

Leadership workshop, one day

# Design an AI-Native Software Factory for Your Organization

A working session for people who decide how engineering works. They do not write code here. They leave with an operating model, an autonomy matrix and a sequenced roadmap.

0:00 to 1:15

## What you already have, unwritten

---

**Goal** Each team's current rules are on paper, and the variation between teams is visible to everyone in the room.

---

**Say**

- Open by asserting that they already have an operating model. It is informal, it differs by team, and nobody has compared the versions.
- Have each team write theirs on a card: what a machine may do unattended, what needs approval, and who grants exceptions.
- Put the cards side by side and let the spread speak. That gap is the reason for the day, and it is more persuasive than any slide.

---

**Run** Nothing technical. Cards on a wall.

---

**Room does**

- Produces one card per team and reads two others aloud.
- Marks every place two teams contradict each other.

---

**Ask** *Where two of your teams disagree, which one is right, and who would decide that today?*

---

1:30 to 3:00

## The autonomy matrix

---

**Goal**                      Their estate is sorted by consequence, with approval placed where the risk actually is.

---

- Say**
- Kill the difficulty axis early, using an example from their own estate rather than a generic one.
  - Run the sort as a group exercise across their real services, and let the arguments happen. The arguments are the product.
  - Push back whenever everything lands in the strictest column. A model where nothing may proceed is a model nobody will follow.
- 

**Run**

```
$ node steps/01-boundary/demo.ts graph
```

Show the worked example once, as a shape to copy, then put it away.

---

- Room does**
- Places every significant service in a column, with a named owner.
  - Agrees who may move something between columns and how that gets recorded.
- 

**Ask**                      *Which service caused the longest argument? That is the one your written policy most needs to cover.*

---

3:45 to 5:00

## Protecting the standard you already have

|           |                                                                                                                                                                                                                                                                                                                                                                                                                                                                    |
|-----------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Goal      | They know which checks must run on every change and which belong before release, and where their current setup fails open.                                                                                                                                                                                                                                                                                                                                         |
| Say       | <ul style="list-style-type: none"><li>• Ask what a machine could produce today that would pass their pipeline and still be wrong. Let the room be uncomfortable.</li><li>• Introduce failing closed as a governance property rather than a technical one, because a missing check is invisible in every report they receive.</li><li>• Explain the independent read in terms they own: an assurance step, with a cost per change they can decide to pay.</li></ul> |
| Run       | <pre>\$ bash steps/04-verification/gates.sh --without-tests</pre>                                                                                                                                                                                                                                                                                                                                                                                                  |
| Room does | <ul style="list-style-type: none"><li>• Sees a missing check produce something that is neither a pass nor a failure.</li><li>• Lists the checks that will be required across every team.</li></ul>                                                                                                                                                                                                                                                                 |
| Ask       | <i>How would you learn, this month, that one of your teams quietly disabled a required check?</i>                                                                                                                                                                                                                                                                                                                                                                  |

5:00 to 5:30

## Who is accountable for a change nobody wrote

---

**Goal** Accountability is assigned explicitly, and the capacity limit is a number rather than a feeling.

---

**Say**

- Put the question bluntly, because it is the one their board will ask them. A machine wrote it, someone approved it, and something went wrong.
- Settle it the same way as any other change: a named engineer approved it and owns it.
- Then give them the constraint that follows. Approval capacity is finite, so the pipeline must stop producing when it is full.

---

**Run**

```
$ node steps/01-boundary/demo.ts capacity
```

---

**Room does**

- Writes the accountability rule in one sentence.
- Sets a review capacity per team and says how it will be measured.

---

**Ask** *How many changes per week can each of your teams genuinely stand behind?*

---

5:30 to 6:00

## Sequencing it

---

|             |                                                                                       |
|-------------|---------------------------------------------------------------------------------------|
| <b>Goal</b> | They leave with an ordered plan whose first step is small enough to happen next week. |
|-------------|---------------------------------------------------------------------------------------|

---

- |            |                                                                                                                                                                                                                                                                                                                                                                                                              |
|------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>Say</b> | <ul style="list-style-type: none"><li>• Insist the first move is boring and cheap. Trust is built by a small thing that works, not by a programme.</li><li>• Have them pick the first workflow by two criteria: it happens often, and getting it wrong is recoverable.</li><li>• Name what they will measure before starting, so the second conversation is about numbers rather than impressions.</li></ul> |
|------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
- 

|            |                                          |
|------------|------------------------------------------|
| <b>Run</b> | Nothing. The roadmap template on screen. |
|------------|------------------------------------------|

---

- |                  |                                                                                                                                                                        |
|------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>Room does</b> | <ul style="list-style-type: none"><li>• Fills in the first three steps with owners and dates.</li><li>• Agrees the baseline they will compare against later.</li></ul> |
|------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
- 

|            |                                                                             |
|------------|-----------------------------------------------------------------------------|
| <b>Ask</b> | <i>What will you be able to show in six weeks, and who needs to see it?</i> |
|------------|-----------------------------------------------------------------------------|

---

2:20 to 2:40

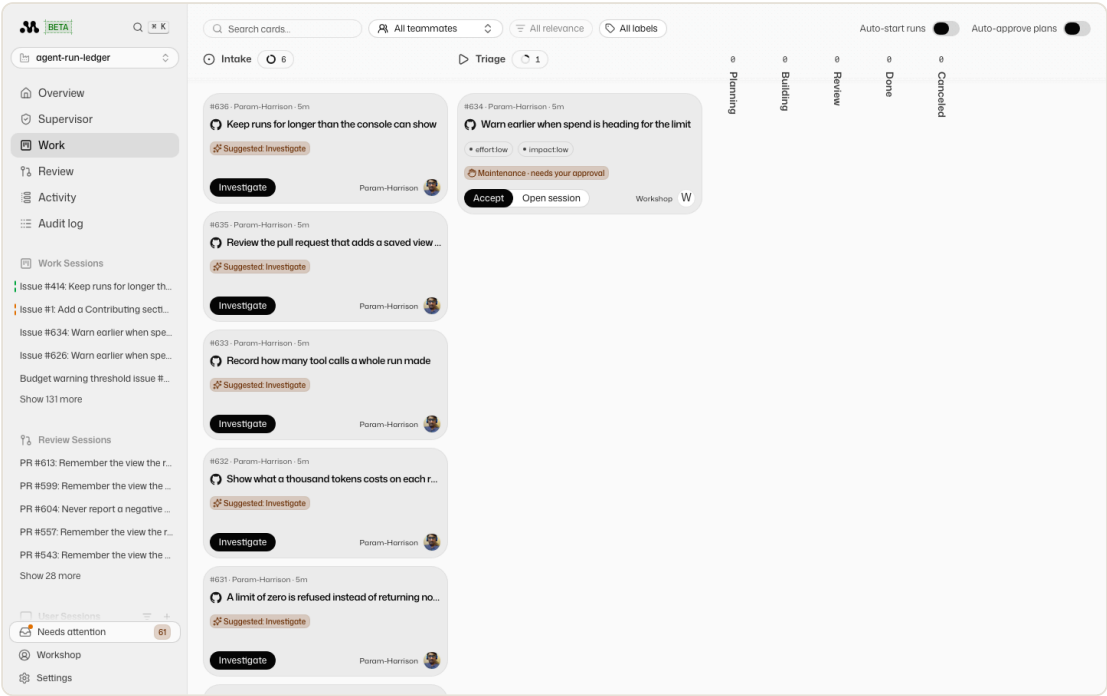
# Four shapes, one afternoon

**Goal** Each leader has chosen a shape for one real team and can defend the cost it imposes.

- Say**
- Switch the shape live and re-run the same item so the room sees the rules doing the work rather than the model.
  - None of the four is correct. Each is a position about who carries the cost of being wrong.
  - Push back on anyone choosing the strictest by default: it has a price, and it is paid in throughput by people who are not in this room.

**Run**

```
$ make profile NAME=solo
$ make profile NAME=enterprise
```



The same item, under the strictest of the four shapes.

**Show on the board** Settings to the two switches to Work to the money item

---

**A shape is a commit, not a setting**

make profile writes the charter and the ownership graph and pushes them, because a run clones from the remote. A shape that exists only on somebody's laptop is a shape no agent will ever read.

**The queue cap is the number to argue about**

Every shape declares how many decisions may wait on a person. That number, not the model, is what decides whether the factory helps or buries the team.

**Nobody in the room pays the cost of the strictest shape**

Pushing everything to refuse feels responsible and is paid for in throughput by people who are not at this table. Ask who they are and whether they agreed.

---

**Room does**

- Picks a shape for one named team and writes the trade-off it accepts.
  - Names the first rule they would relax after a quarter of evidence.
- 

**Ask**

*Who in your organisation currently pays for a change being wrong, and do they know?*

---

Concepts: the six jobs, a written policy, grading autonomy, paths off limits, capacity, accountability, measuring, ownership.